PeerScope: Paper Evaluation and Auditable Reviewer Recommendation with Local AI

Mula Sreebindu
Roll No. 23K91A05G3
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[EMAIL]

Puramshetty Vishnu Vardhan
Roll No. 23K91A05K2
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[EMAIL]

K Sampoorna
Roll No. 24K95A0512
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[EMAIL]

Korsa Shivashankar
Roll No. 24K95A0515
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[EMAIL]

M Anupama
[DESIGNATION] (project guide)
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[EMAIL]

***Abstract*---**Journals and conferences receive more submissions than their reviewers can absorb, and both first-pass screening and the choice of reviewer remain largely manual. This paper presents PeerScope, a local web application that turns a research-paper PDF into a preliminary, clearly caveated assessment and an auditable ranking of candidate reviewers. The paper is parsed into sections and structural signals, a TF-IDF logistic-regression model supplies a historical decision signal, a locally run Qwen2.5-1.5B model drafts a critique, and related work is retrieved from OpenAlex. Reviewers are ranked by a transparent score that combines sentence-embedding similarity, keyword and method overlap, availability and a conflict penalty. On 40 probe abstracts written by the team and six demo reviewers, the hybrid score ranks an acceptable reviewer first in 95% of cases (75% with an offline TF-IDF fallback). The decision model, trained on synthetic labels, reaches 0.55 accuracy, close to the 0.56 ceiling of those labels, so it demonstrates the pipeline and does not predict real decisions. The results are not comparable with earlier work on real conference data.

***Index Terms*---**artificial intelligence, conflict of interest, natural language processing, peer review, reviewer recommendation, semantic similarity, sentence embeddings.

I. INTRODUCTION

Scholarly publishing is limited by reviewer capacity. When two independent NeurIPS 2021 committees reviewed the same papers they disagreed on about 23 % of accept-or-reject outcomes [2], and each submission still has to be matched to qualified reviewers, a task studied for about thirty years as the reviewer assignment problem [3]. In most venues an editor shortlists reviewers from personal knowledge or keyword search, which favours people who share vocabulary rather than expertise, and screens each submission by hand before any reviewer sees it.

Research has served the two halves of this workload separately. Assignment methods compute paper-reviewer affinity from text, from topic models or from learned suitability scores [4], [5], [6]; automated-review work, surveyed in [7], predicts decisions or writes review text [8], [9], [10], [11]. The base paper [1] argues that the affinity score is the hard part and that dense embeddings are hard to audit, so it restricts language models to extracting keywords and keeps a full audit trail. It does not assess the submission itself, and it left conflict of interest and reviewer capacity out of its experiments. Language-model reviewers also carry risks: hidden instructions in a manuscript steer them [12], [13].

PeerScope closes the first gap on a small scale. It is a local application, with no cloud service for the paper text, that gives an editor both a preliminary assessment of an uploaded PDF and a reviewer ranking whose every term is visible. The contributions are: (1) an end-to-end pipeline from PDF to assessment to ranked reviewers, with a stored history and JSON and PDF export; (2) an auditable score that puts embedding similarity, keyword and method overlap, availability and a conflict penalty in one explicit formula; (3) a measured evaluation, including a baseline study, a probe-set test of ranking quality, a conflict-injection test and latency, reported together with what it cannot show; and (4) a record of the defects found in testing. Section II reviews related work, Section III the method, Section IV the results and Section V concludes.

A. System Hardware

Every measurement used one desktop PC: Intel Core i5-12500 with 15.7 GB of RAM, no graphics processor and Windows 11. Language-model generation therefore ran on the CPU in 32-bit floating point.

B. Software Requirements

Python 3.11.9; FastAPI 0.120.4 with SQLite through SQLAlchemy 2.1.6; PyMuPDF 2.0 for text extraction; scikit-learn 1.9.1 [14]; sentence-transformers 6.2.0 with all-MiniLM-L6-v2 [15]; PyTorch 2.14.1 and Transformers 5.20.2 running Qwen2.5-1.5B-Instruct [16]; the OpenAlex API [17] for literature; and a React and Vite interface for the browser.

II. LITERATURE SURVEY

Base paper. Bagheri et al. [1] treat reviewer assignment as an affinity-construction problem. Producer language models each return six to twelve keyphrases for a paper view (title, abstract summary or abstract) or for each prior publication of a reviewer; a stronger judge model picks the best list without rewriting it; deterministic normalisation follows; and every stage stores prompts, model identifiers

and score matrices as an audit trail. Matchers include TF-IDF cosine, keyword Jaccard, an SBERT bi-encoder (all-MiniLM-L6-v2) with a cross-encoder re-ranker, and keyword knowledge graphs. The data are 663 papers from 85 computer-science conferences with 1,989 historical assignments and 524 reviewer profiles built from Semantic Scholar. The best mean reciprocal rank at three (MRR@3) is 0.7778 on titles (two-stage re-ranking) and 0.7843 on abstracts (graph edge Jaccard); the best precision at three is 0.4545. Most differences between text encoders were not statistically significant. The authors state that historical assignments are an imperfect label, that no human expert judgements were used, and that conflict of interest and capacity could not be benchmarked.

Reviewer assignment. Charlin et al. [4] learn suitability scores from a few provided scores and then solve an integer program; Mimno and McCallum [5] compare language-model and author-persona-topic models against human relevance judgements; Chain-of-Factors [6] chains semantic, topic and citation factors in one instruction-tuned model; and Jecmen et al. [18] randomise assignments to limit manipulation. A survey [3] organises the field into candidate database, matching degree and assignment optimisation.

Representations. Sentence-BERT [19] makes sentence embeddings comparable by cosine similarity (finding the closest pair among 10,000 sentences falls from about 65 hours to about 5 seconds), and MiniLM [15] is the compact model family used here and in the base paper.

Automated review. PeerRead [9] offers 14.7 thousand drafts with decisions and 10.7 thousand reviews, where simple models cut error by up to 21 % against the majority baseline. Debate graphs over simulated reviewer-author exchanges (ReViewGraph) [8] reach 66-72 % accuracy on ICLR 2023-2025 decisions. Generated reviews cover more aspects than human ones but are less constructive [10]; GPT-4 feedback overlaps with human reviewers about as much as two humans overlap with each other [11]. Hidden instructions were followed in 78 % to 86 % of trials with two chatbots [12], and simple prompt injection raised acceptance scores up to 100 % [13].

TABLE I
REPRESENTATIVE RELATED WORK

Work	Approach	Data	Reported result	Limitation
[1]	LLM keywords, audit trail, SBERT and graph matchers	663 papers, 524 reviewers	MRR@3 up to 0.78	No paper assessment; no COI test
[4]	Learned suitability + integer program	Two conferences	Better than baselines	Needs provided scores
[5]	Author-persona-topic model	Rexa, human judgements	APT best of three	Topic model only
[9]	Decision and review dataset	14.7K drafts	Up to 21 % error cut	Old text models
[11]	GPT-4 feedback on PDFs	Nature, ICLR papers	31-39 % overlap	No reviewer matching
[8]	Debate graph + GNN	ICLR 2023-25	66-72 % accuracy	Decisions only

Gap. Assignment research scores affinity but does not look at the submission; review research looks at the submission but does not choose reviewers; the base paper leaves availability and conflict of interest outside its experiments; and no work in this survey delivers both halves, locally and with a visible score, as one application. PeerScope targets that combination at demonstration scale; it does not attempt to match the base paper's data or its keyword-induction ensemble.

III. METHODOLOGY

Architecture. Fig. 1 shows the three tiers. A React single-page application calls a FastAPI service on the same machine; the service runs a set of plain Python service modules and keeps two SQLite tables, one for reviewers and one for stored analyses, plus the trained model files. There is no login and nothing is sent to a cloud service apart from a title query to OpenAlex.

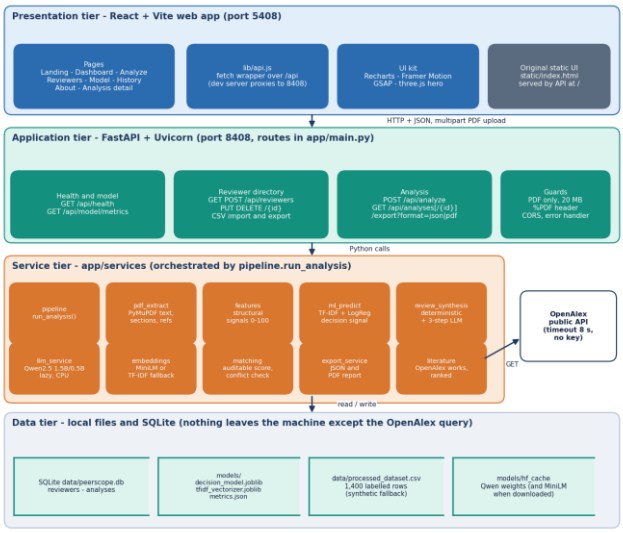


Fig. 1. PeerScope architecture: browser tier, FastAPI service with its service modules, and file and database storage.

Main flow. Fig. 2 follows one upload (at most 20 MB, with a PDF header check). PyMuPDF extracts text page by page; the text is normalised, the title is taken from the first plausible line, nine section types are found by pattern matching and references are parsed. Frequency keywords, a 21-term method lexicon and 18 topic hints describe the paper, and four structural signals (section completeness A, evidence coverage B, method clarity C and presentation D) combine as

$$S_{struct} = 0.35\,A + 0.25\,B + 0.25\,C + 0.15\,D. \qquad (1)$$

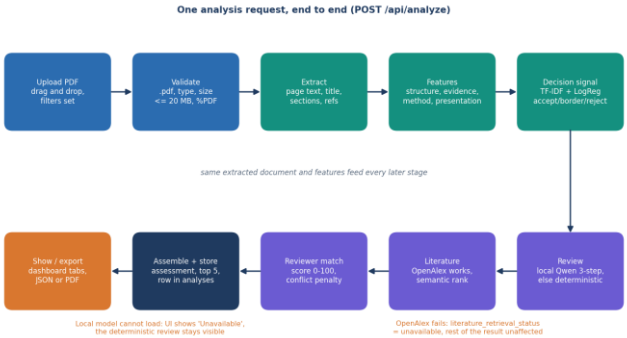


Fig. 2. Main flow: extraction, signals, decision model, review, literature and reviewer ranking.

Decision signal. A TF-IDF representation (8,000 word 1-2-grams and 4,000 character 3-5-grams) feeds a logistic regression with balanced class weights [14] that outputs probabilities for accept, borderline and reject, p(k|x) = softmax(Wx + b). No public peer-review corpus could be fetched, so the model is trained on a synthetic set of 1,400 papers generated by the project (80/20 stratified split, seed 42). Its labels are drawn from the same strength and weakness phrases that appear in the text, and the borderline class arises from both groups, so no text classifier can exceed about 0.56 accuracy on it.

Review. Deterministic rules always produce strengths, concerns and recommendations from the signals. If it can be loaded, a locally run Qwen2.5-1.5B-Instruct model [16] adds a three-step analyse, critique and synthesise text; if not, the interface says so and shows only the deterministic part. Related work is searched in OpenAlex by title and ranked by embedding similarity.

Reviewer matching. Paper and reviewer profile texts are embedded with all-MiniLM-L6-v2, with a TF-IDF fallback when the model cannot be loaded. The score of reviewer r for paper p is

$$S(p, r) = 100 \cdot clip(0.55\ s_{emb} + 0.20\ s_{kw} + 0.15\ s_{meth} + 0.10\ s_{avail} - 0.60\ c), \quad (2)$$

where s_emb is the cosine similarity of the embeddings, s_kw and s_meth are overlap scores between paper and reviewer keywords, and between methods and topics, s_avail = 1 − load/capacity, and c is 1 when a conflict is detected and 0 otherwise. A conflict is a listed conflict term, the reviewer's full name or the reviewer's affiliation appearing in the first 3,000 characters of the paper. Scores of at least 65, 45 and 25 are labelled Strong, Good and Possible. Every recommendation shows all five terms, the matching words and a one-line explanation, which is the auditability inherited from the base paper's principle [1]. The directory holds six clearly labelled demo reviewers; the ranking is only a ranking, with no global assignment optimiser.

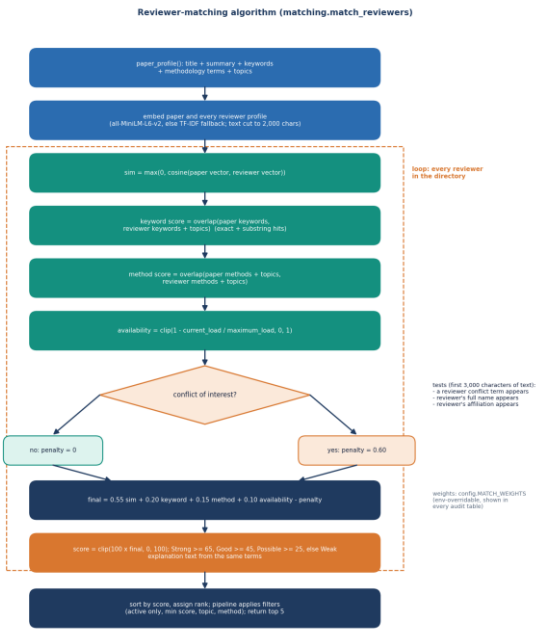


Fig. 3. Core algorithm: profile construction, embedding similarity, overlap, availability, conflict detection, weighted score and ranking.

Implementation. The service exposes twelve endpoints: health, model metrics, create, list, update, delete, import and export of reviewers, analysis upload, history, retrieval and export as JSON or PDF. Uploads are rejected with a clear message when the extension or content type is wrong, the file is empty, the PDF header is missing (HTTP 400) or the size exceeds 20 MB (HTTP 413); reviewer fields are length-limited by Pydantic schemas (HTTP 422), unknown identifiers return 404, and an unexpected error returns a generic 500 without internal details. A stored analysis keeps the full result as JSON. The code base is 1,449 lines of Python for the service, 6,926 lines for the eight-page interface and 114 lines of tests.

Evaluation protocol. The decision model is evaluated on the 280 held-out synthetic papers, against majority-class, naive Bayes and unweighted logistic-regression baselines, and over five different splits. Ranking is evaluated on 40 abstracts written by the team, each labelled with the demo reviewer or reviewers whose profile fits; top-1, top-3 and mean reciprocal rank are reported for four score configurations. Fifteen conflict links were injected into a paper (12 literal, 3 paraphrased) and 240 clean paper-reviewer pairs were checked for false flags. Four real PDFs, including the base paper, were run through the pipeline, and latency was measured stage by stage and over HTTP.

Verification. 7 pytest tests (extraction, features, matching and conflict, model service, health, reviewer create-read-update-delete and a full analysis) and a 24-check smoke test against a running server (validation, CRUD, CSV, analysis, history, export, CORS) pass in 34 s with the language model switched off.

IV. RESULTS

Test conditions. All numbers come from one machine (Section I-A). The decision-model figures use the 280-paper synthetic test split; the ranking figures use the 40-abstract probe set and the six demo reviewers, whose profiles and labels were written by the same team, so they are optimistic and are not independent expert judgements. Table II lists the main results.

TABLE II
MEASURED RESULTS

Evaluation	Setting	Result
Decision model, 280 test papers	accuracy / macro F1	0.55 / 0.509
	macro precision / recall	0.53 / 0.65
	ROC-AUC (one vs rest)	0.771
	5 splits, accuracy	0.545 +/- 0.028
	accept vs reject only	93% (145/156)
	baselines, accuracy	majority 0.44, NB 0.46
Ranking, 40 probes	hybrid, MiniLM: top-1 / top-3 / MRR	0.95 / 0.97 / 0.96
	semantic only, MiniLM	0.97 / 1.00 / 0.98
	lexical only	0.62 / 0.82 / 0.74
	hybrid, TF-IDF fallback	0.75 / 0.93 / 0.84
	random choice, top-1 / top-3	0.18 / 0.53
Conflict check	literal / paraphrased links flagged	12/12 / 0/3
	false flags, clean pairs	0/240

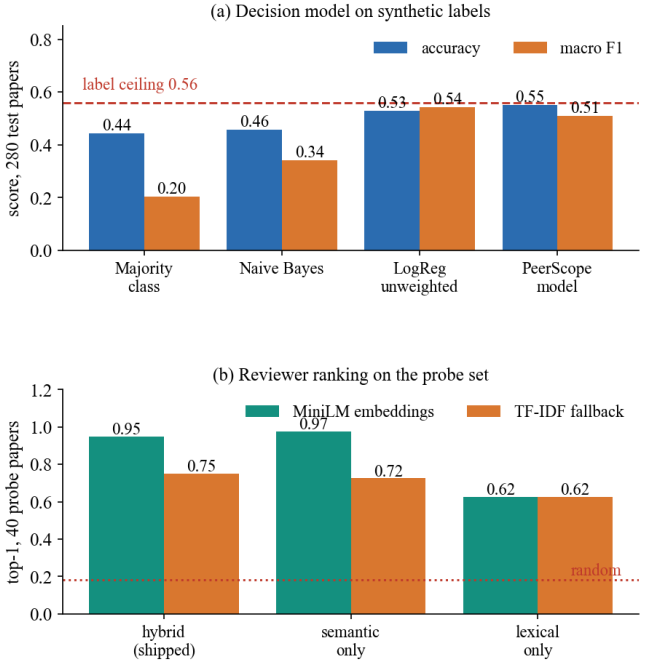


Fig. 4. (a) Decision model against baselines, with the ceiling set by the synthetic labels. (b) Top-1 ranking accuracy on the probe set for three score configurations with two embedders.

Decision model. Accuracy 0.55 is only 0.01 below the 0.56 ceiling of the labels, and it is 0.11 above the majority baseline (0.44). Accept and reject are found (recall 0.96 and 0.90) but borderline is not (recall 0.07): 45 of 124 borderline papers were called accept and 70 reject. Over five splits accuracy varies from 0.51 to 0.59. On the four real PDFs, including the base paper, the model answered borderline every time with confidence 0.55 to 0.66, so its signal carries no evidence about real papers and the interface labels it as synthetic.

Ranking. With sentence embeddings the hybrid score ranks an acceptable reviewer first for 95% of the probes, against 18% for a random choice, and in the top three for 97.5%. Semantic similarity alone is slightly better (97.5%), lexical overlap alone is clearly worse (62.5%), and the offline fallback falls to 75%, because its vocabulary is fitted once on the first request and then frozen. The hybrid terms therefore do not improve ranking on this probe set; their value is the explanation and the availability and conflict terms, which the probe set does not test. Ranking is fast with six reviewers (86 ms) but every profile is re-embedded on every request, so 500 reviewers take 5.2 s.

Conflicts. All 12 literal links (conflict terms, affiliations, full names) were flagged and the penalty moved the reviewer to the bottom of the ranking; none of the 3 paraphrased links (a reworded affiliation, a first name only) was found, and there were no false flags in 240 clean pairs. Detection is string matching, not a co-authorship check.

Speed. The deterministic path needs 1.3 s per analysis over HTTP (median of 8); reading and ranking need well under a second apart from OpenAlex (2.3 s). With the language model on, one analysis took 534 to 946 s, of which the three-step review alone took 509 s for 1,410 generated tokens (about 3 tokens per second).

Confidence and score separation. The decision model's confidence is informative but weak: papers with confidence of at least 0.6 are 65% of the test set at 0.58 accuracy, at least 0.7 are 29% at 0.70, and the 11 papers above 0.8 reach 0.73; no setting approaches the accuracy a clean label would allow. For ranking, the first-ranked reviewer leads the second by 12.6 points on average over the probe set, so a Strong or Good band is usually not reached for short abstracts, and the bands should be read as relative to the pool.

TABLE III
REAL PDFS THROUGH THE PIPELINE

Paper	Pages	Words	Struct.	Sections	Decision
Demo fixture	3	431	68.1	9/9	borderline 0.66
Survey [7]	30	19,877	68.5	8/9	borderline 0.55
ReViewGraph [8]	17	11,562	65.2	7/9	borderline 0.58
Base paper [1]	30	21,708	75.9	6/9	borderline 0.55

Real papers. Table III shows that the extraction and signals behave sensibly on a three-page fixture and on three real 17- to 30-page papers: 6 to 9 of the 9 expected sections are found and the structural score stays between 65 and 76. The score is a heuristic with unvalidated weights and cannot rank papers by quality; it mainly flags missing sections, short methodology text and sparse references. The fixture is a generated document, not a real submission.



Fig. 5. Output screen: analysis of a real 17-page paper, showing the structural score, concerns, top reviewer, decision signal and the language-model status.

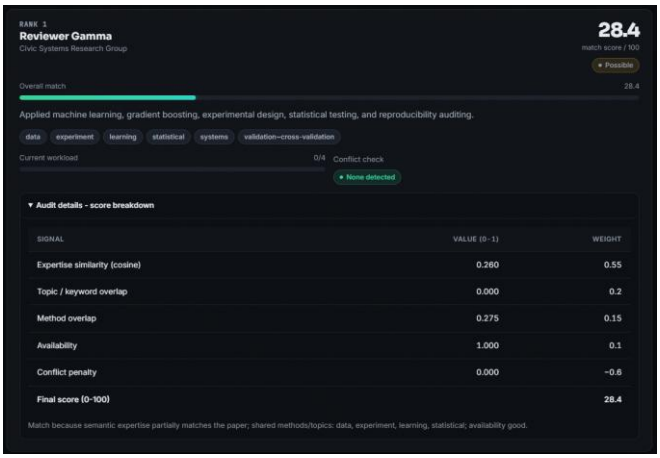


Fig. 6. Output screen: top-ranked reviewer with the auditable score breakdown (expertise similarity 0.260, method overlap 0.275, availability 1.0).

Failure cases. (1) For the 17-page paper in Fig. 5 and 6 the best reviewer scored only 28.4 of 100 and two reviewers tied; the NLP reviewer ranked fourth, because the demo pool has no reviewer on language-model reviewing. Ranks also skip a number when an inactive reviewer is filtered out afterwards. (2) The two misses on the probe set were a malware-evasion abstract assigned to the applied-statistics reviewer instead of the security reviewer and a cross-area log-analysis abstract. (3) For the base paper's PDF the introduction was not detected, because the heading is a roman numeral, and the title heuristic returned the DOI line. (4) The title of the survey PDF contains a question mark, which made OpenAlex answer HTTP 400, so related work was reported unavailable. (5) The language-model text for the 17-page paper misdescribes its datasets, which is why it is shown as generated text that can contain errors. (6) The coverage chart's novelty value is a fixed 55; novelty is not measured.

Per-area results and uncertainty. The probe set has six abstracts for each of six areas plus four cross-area abstracts with two acceptable reviewers. With sentence embeddings the hybrid score is right for all six abstracts in five areas and five of six in security, and for 3 of 4 cross-area abstracts; the fallback embedder drops to 3 of 6 in vision and 3 of 6 in security. With 40 probes the 95 % binomial interval around 95% is about +/-7 points, so the difference between the hybrid and the semantic-only score (one abstract) is within noise, while the gap to lexical-only is not.

Threats to validity. The reviewer profiles and the probe abstracts were written by the same team, which favours embedding similarity; the six profiles are far apart in topic, so the task is easier than a real pool of hundreds of overlapping researchers; the decision model was trained and tested on data of one synthetic kind; the language-model text was not rated by human reviewers; no user study was run; and every timing comes from one CPU-only machine. These limits are why the numbers above are read as evidence that the pipeline works, not as a benchmark.

Comparison with the base paper. The comparison is context only. The base paper reports MRR@3 up to 0.78 on 663 real conference papers and 524 reviewers with historical assignments as labels, whereas PeerScope reports MRR 0.96 on 40 team-written abstracts and six demo reviewers, and adds paper assessment, availability and conflict terms that the base paper did not benchmark. Data, task size, labels and metric (rank of the first acceptable reviewer, not MRR@3 over three) differ, so no claim of superiority is made.

V. CONCLUSION

PeerScope delivers a working local pipeline that turns a PDF into an assessment and an auditable reviewer ranking. Against the six objectives: corpus building is partial (stored analyses and a six-profile reviewer directory, no real publication corpus); topic, method and keyword extraction and embedding-based reviewer vectors are done with simple lexicons and profile text; ranking is done and measured; the preliminary assessment is done except for novelty; and evaluation against expert judgement is only partial, using the team's own labels. The key finding is that the explicit hybrid score ranks well on the probe set (95% top-1) but a plain embedding does as well, and that the decision model's 0.55 accuracy reflects the synthetic labels, not real peer-review decisions. Other limits are the six demo reviewers, literal-match conflict detection, minutes-long CPU generation, an unvalidated structural weighting and the defects listed above.

Future work: evaluate ranking on real conference assignments with independent expert labels; train the decision model on real review data such as PeerRead [9]; build reviewer profiles from publication records; add a co-authorship conflict check and a capacity-aware assignment optimiser; cache reviewer embeddings; sanitise the literature query and recognise roman-numeral headings; and defend the language-model step against hidden instructions in the PDF.

REFERENCES

[1] F. Bagheri, D. Buscaldi and D. Reforgiato Recupero, "LLM-assisted reviewer assignment via auditable expertise matching," *IEEE Access*, vol. 14, pp. 127834-127863, 2026, doi: 10.1109/ACCESS.2026.3724622.

[2] A. Beygelzimer, Y. N. Dauphin, P. Liang and J. W. Vaughan, "Has the machine learning review process become more arbitrary as the field has grown? The NeurIPS 2021 consistency experiment," arXiv:2306.03262, Jun. 2023.

[3] X. Zhao and Y. Zhang, "Reviewer assignment algorithms for peer review automation: a survey," *Inf. Process. Manage.*, vol. 59, no. 5, Art. no. 103028, Sep. 2022, doi: 10.1016/j.ipm.2022.103028.

[4] L. Charlin, R. S. Zemel and C. Boutilier, "A framework for optimizing paper matching," arXiv:1202.3706, Feb. 2012.

[5] D. Mimno and A. McCallum, "Expertise modeling for matching papers with reviewers," in *Proc. 13th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2007, pp. 500-509, doi: 10.1145/1281192.1281247.

[6] Y. Zhang, Y. Shen, S. Kang, X. Chen, B. Jin and J. Han, "Chain-of-factors paper-reviewer matching," in *Proc. ACM Web Conf. 2025*, Apr. 2025, pp. 1901-1910, doi: 10.1145/3696410.3714708.

[7] S. Wu, O. Jiang, Y. Zhao, T. Hu, Y. Ma, K. Zhang, M. Patwardhan and A. Cohan, "Can AI be a good peer reviewer? A survey of peer review process, evaluation, and the future," arXiv:2604.27924, Apr. 2026.

[8] S. Li, L. Fan, Y. Lin, Z. Li, X. Wei, S. Ni, H. Alinejad-Rokny and M. Yang, "Automatic paper reviewing with heterogeneous graph reasoning over LLM-simulated reviewer-author debates," arXiv:2511.08317, Nov. 2025 (AAAI 2026 copyright notice on the PDF).

[9] D. Kang, W. Ammar, B. Dalvi, M. van Zuylen, S. Kohlmeier, E. Hovy and R. Schwartz, "A dataset of peer reviews (PeerRead): collection, insights and NLP applications," in *Proc. NAACL-HLT*, 2018, pp. 1647-1661, doi: 10.18653/v1/N18-1149.

[10] W. Yuan, P. Liu and G. Neubig, "Can we automate scientific reviewing?," *J. Artif. Intell. Res.*, vol. 75, pp. 171-212, 2022, doi: 10.1613/jair.1.12862.

[11] W. Liang et al., "Can large language models provide useful feedback on research papers? A large-scale empirical analysis," *NEJM AI*, vol. 1, no. 8, Jul. 2024, doi: 10.1056/AIoa2400196.

[12] F. Torrielli, S. Locci, A. Rapp and L. Di Caro, "How to get your paper accepted by an AI reviewer: indirect prompt injection in peer review," Research Square preprint, Jan. 2026, doi: 10.21203/rs.3.rs-8432945/v1. (Published as "Exploiting large language models in peer review: indirect prompt injection attacks and integrity probes," *Scientometrics*, vol. 131, no. 7, pp. 4889-4951, 2026, doi: 10.1007/s11192-026-05695-x.)

[13] J. Keuper, "Prompt injection attacks on LLM generated reviews of scientific publications," arXiv:2509.10248, Sep. 2025.

[14] F. Pedregosa et al., "Scikit-learn: machine learning in Python," *J. Mach. Learn. Res.*, 2011 (arXiv:1201.0490).

[15] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang and M. Zhou, "MiniLM: deep self-attention distillation for task-agnostic compression of pre-trained transformers," arXiv:2002.10957, Feb. 2020.

[16] A. Yang et al. (Qwen Team), "Qwen2.5 technical report," arXiv:2412.15115, Dec. 2024.

[17] J. Priem, H. Piwowar and R. Orr, "OpenAlex: a fully-open index of scholarly works, authors, venues, institutions, and concepts," arXiv:2205.01833, 2022.

[18] S. Jecmen, H. Zhang, R. Liu, N. B. Shah, V. Conitzer and F. Fang, "Mitigating manipulation in peer review via randomized reviewer assignments," arXiv:2006.16437, Jun. 2020.

[19] N. Reimers and I. Gurevych, "Sentence-BERT: sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3980-3990, doi: 10.18653/v1/D19-1410.